LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 05 – RELASI ANTAR KELAS

MUHAMMAD ‘IZZUDDIN NABIL

103132400032

PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pemodelan berorientasi objek menggunakan Diagram Kelas untuk menggambarkan komponen-komponen sistem beserta relasinya. Secara visual, diagram ini digambarkan dalam bentuk kotak yang terbagi menjadi tiga bagian utama: nama kelas di baris pertama, daftar atribut di baris kedua, dan daftar operasi atau *method* di baris ketiga. Diagram kelas juga menggunakan simbol khusus untuk menandakan tingkat akses (*access modifier*), seperti tanda + untuk akses *public*, tanda - untuk *private*, dan tanda # untuk *protected*.

Dalam penerapannya, kelas-kelas ini saling berinteraksi melalui berbagai jenis Relasi Antar Kelas. Bentuk relasi yang paling longgar adalah Asosiasi, di mana dua kelas yang mandiri saling berkomunikasi, misalnya dengan menggunakan objek dari kelas lain sebagai parameter pada sebuah *method* tanpa adanya status kepemilikan. Tingkat relasi yang lebih erat adalah Agregasi, yang mewakili hubungan keseluruhan dan bagian (*part-whole*). Pada agregasi, kelas bagian (*part*) merupakan komponen penyusun kelas utama (*whole*), namun objek bagian tersebut masih dapat diciptakan dan berdiri sendiri meskipun kelas utamanya tidak ada.

Bentuk hubungan *part-whole* yang paling kuat dan ketat disebut dengan Komposisi. Pada relasi komposisi, keberadaan kelas bagian mutlak bergantung pada kelas utamanya. Biasanya, objek penyusun ini diinstansiasi langsung di dalam kelas utama, sehingga apabila objek utama (keseluruhan) dihapus atau musnah, maka seluruh objek bagian yang menyusunnya akan otomatis ikut musnah.

B. Guided

1. Asosiasi

```java
package guided.guided1;

public class Mahasiswa {
    private String nim;
    private String nama;
    private int nilai;

    // Setter
    public void setNama(String nama) {
        this.nama = nama;
    }

    public void setNim(String nim) {
        this.nim = nim;
    }

    public void setNilai(int nilai){
        this.nilai = nilai;
```

```
    }

    // Getter
    public String getNama() {
        return this.nama;
    }

    public String getNim() {
        return this.nim;
    }

    public int getNilai(){
        return this.nilai;
    }
}
```

```
package guided.guided1;

public class Dosen {
    private String kodeDosen;
    private String namaDosen;

    // Setter
    public void setKodeDosen(String kodeDosen) {
        this.kodeDosen = kodeDosen;
    }

    public void setNamaDosen(String namaDosen) {
        this.namaDosen = namaDosen;
    }

    // Getter
    public String getKodeDosen() {
        return this.kodeDosen;
    }

    public String getNamaDosen() {
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa s, int nilai){
    // ini asosiasi, method milik class Student dipanggil di class
Dosen,
    // tp objek Student tidak menjadi atribut dr class Lecture
```

```
    s.setNilai(nilai);
    }

    public int getScore(Mahasiswa s){
    // ini asosiasi, method milik class Student dipanggil di class
Lecture,
    // tp objek Student tidak menjadi atribut dr class Lecture
    return s.getNilai();
    }

    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();
        m.setNim("130118383");
        m.setNama("Budi");
        Dosen d = new Dosen();
        d.giveScore(m, 90);
        System.out.println("Nim :" + m.getNim());
        System.out.println("Nama :" + m.getNama());
        System.out.println("Nilai :" + d.getScore(m));
    }
}
```

Output :

```
PS D:\Praktikum PBO\modul-05-relasi-antar-kelas>  &
ionMessages' '-cp' 'C:\Users\Lenovo\AppData\Roaming\(
dt_ws\modul-05-relasi-antar-kelas_e3d4adcb\bin' 'guic
Nim :130118383
Nama :Budi
Nilai :90
```

2. Agregasi

```
package guided.agregasi;

public class Mahasiswa {
    private String NIM, Nama;

    public Mahasiswa(String NIM, String Nama){
        this.NIM = NIM;
        this.Nama = Nama;
    }

    // Getter
    public String getNIM(){
        return NIM;
```

```
    }

    public String getNama(){
        return Nama;
    }

    // Setter
    public void setNIM(String NIM){
        this.NIM = NIM;
    }

    public void setNama(String Nama){
        this.Nama = Nama;
    }
}
```

```
package guided.agregasi;

public class Jurusan {
    private static int jumlahMahasiswa;
    private String kodeJurusan, namaJurusan;
    // menyimpan objek dari class lain di dalam class nya
    private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];

    // Constructor
    public Jurusan(String kodeJurusan, String namaJurusan){
        this.kodeJurusan = kodeJurusan;
        this.namaJurusan = namaJurusan;

    }

    // Behavior
    public void addMahasiswa(Mahasiswa m){
        this.daftarMahasiswa[jumlahMahasiswa] = m;
        jumlahMahasiswa++;
    }

    public void displayMahasiswa(){
        System.out.println("Kode Jurusan: " + this.kodeJurusan);
        System.out.println("Nama Jurusan: " + this.namaJurusan);

        System.out.println("Daftar Mahasiswa: ");

        for (int i = 0; i < jumlahMahasiswa; i++) {
```

```
            System.out.println(i+1 + ". " +
daftarMahasiswa[i].getNIM() + " " + daftarMahasiswa[i].getNama());
        }
    }

    // Setter
    public void setKodeJurusan(String kodeJurusan){
        this.kodeJurusan = kodeJurusan;
    }

    public void setNamaJurusan(String namaJurusan){
        this.namaJurusan = namaJurusan;
    }

    // Getter
    public String getKodeJurusan(){
        return kodeJurusan;
    }

    public String getNamaJurusan(){
        return namaJurusan;
    }

}
```

Main gabungan

```
package src;

import guided.agregasi.*;
import guided.komposisi.Komputer;

public class Main {
    public static void main(String[] args) {
        runGuided2();
        runGuided3();
    }

    public static void runGuided1() {
    }

    public static void runGuided2() {
        Mahasiswa m1 = new Mahasiswa("103112400148", "Rendhi");
        Mahasiswa m2 = new Mahasiswa("103112400149", "John");
```

```
        Jurusan prodi = new Jurusan("SD", "Sains Data");

        prodi.addMahasiswa(m1);
        prodi.addMahasiswa(m2);

        prodi.displayMahasiswa();

    }

    public static void runGuided3() {
        Komputer pc = new Komputer("Intel Core i5", 3200, "LG 24
Inch", "Logitech", "Wireless");
        System.out.println("");
        pc.displaySpecKomputer();
    }
}
```

Output :

```
PS D:\Praktikum PBO\modul-05-relasi-antar-kelas>  d:;
Java\jdk-17\bin\java.exe' '-XX:+ShowCodeDetailsInExcep
eStorage\479783380518d35d0662f1a127f9c8f1\redhat.java\
Nama Jurusan: Sains Data
Daftar Mahasiswa:
1. 103112400148 Rendhi
2. 103112400149 John
```

3. Komposisi

```
package guided.komposisi;

public class Monitor {
    private String brand;

    public Monitor(String brand) {
        this.brand = brand;
    }

    public void DisplaySpec() {
        System.out.println(this.brand);
    }
}
```

```
package guided.komposisi;
```

```java
public class Mouse {
    private String brand, tipe;

    public Mouse(String brand, String tipe) {
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec() {
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

```java
package guided.komposisi;

public class Processor {
    private String brand;
    private int kecepatan;

    // constructor
    public Processor(String brand, int kecepatan) {
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    // behaviour
    public void displaySpec() {
        System.out.println(this.brand + ", " + this.kecepatan);
    }
}
```

```java
package guided.komposisi;

public class Komputer {
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String
brandMonitor, String brandMouse, String tipeMouse) {
        this.processor = new Processor(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
```

```
    }

    public void displaySpecKomputer() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
        this.processor.displaySpec();
        this.monitor.DisplaySpec();
        this.mouse.displaySpec();
    }
}
```

Main gabungan

```
package src;

import guided.agregasi.*;
import guided.komposisi.Komputer;

public class Main {
    public static void main(String[] args) {
        runGuided2();
        runGuided3();
    }

    public static void runGuided1() {
    }

    public static void runGuided2() {
        Mahasiswa m1 = new Mahasiswa("103112400148", "Rendhi");
        Mahasiswa m2 = new Mahasiswa("103112400149", "John");

        Jurusan prodi = new Jurusan("SD", "Sains Data");

        prodi.addMahasiswa(m1);
        prodi.addMahasiswa(m2);

        prodi.displayMahasiswa();

    }

    public static void runGuided3() {
        Komputer pc = new Komputer("Intel Core i5", 3200, "LG 24
Inch", "Logitech", "Wireless");
        System.out.println("");
        pc.displaySpecKomputer();
    }
```

```
}
```

Output :

```
=== SPESIFIKASI KOMPUTER ===
Intel Core i5, 3200
LG 24 Inch
Logitech, Wireless
```

C. Unguided

1. Soal Unguided 1

Sistem Manajemen Pesanan Cafe

Sebuah cafe membutuhkan sistem sederhana untuk mencatat dan memproses pesanan pelanggan. Dalam sistem ini, relasi antar kelas dirancang untuk mencerminkan kondisi nyata operasional cafe.

Pengerjaan soal ini dibagi menjadi 3 bagian tugas utama yang masing-masing melatih satu jenis relasi antar kelas, diikuti dengan pengujian pada kelas utama (Main).

Jawaban :

Rincian Harga

```
package unguided;

public class RincianHarga {
    private double hargaDasar;
    private double pajak;

    // Constructor
    public RincianHarga(double hargaDasar) {
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 0.1;
    }

    // Behavior
    public double getHargaTotal() {
        return this.hargaDasar + this.pajak;
    }

    // Getter
    public double getHargaDasar() {
        return this.hargaDasar;
    }

    public double getPajak() {
```

```
        return this.pajak;
    }

    // Setter
    public void setHargaDasar(double hargaDasar) {
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 0.1;
    }

    public void setPajak(double pajak) {
        this.pajak = pajak;
    }
}
```

Item Pesanan

```
package unguided;

public class ItemPesanan {
    private String namaItem;
    private RincianHarga rincian;

    // Constructor
    public ItemPesanan(String namaItem, double hargaDasar) {
        this.namaItem = namaItem;
        this.rincian = new RincianHarga(hargaDasar);
    }

    // Getter
    public String getNamaItem() {
        return this.namaItem;
    }

    public RincianHarga getRincian() {
        return this.rincian;
    }

    // Setter
    public void setNamaItem(String namaItem) {
        this.namaItem = namaItem;
    }

    public void setRincian(RincianHarga rincian) {
        this.rincian = rincian;
    }
```

```
}
```

Pesanan

```
package unguided;

public class Pesanan {
    private int nomorMeja;
    private ItemPesanan[] daftarItem;
    private int jumlahItem;

    // Constructor
    public Pesanan(int nomorMeja) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[5];
        this.jumlahItem = 0;
    }

    public Pesanan(int nomorMeja, int kapasitas) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitas];
        this.jumlahItem = 0;
    }

    // Behavior
    public void tambahItem(ItemPesanan item) {
        if (this.jumlahItem < this.daftarItem.length) {
            this.daftarItem[this.jumlahItem] = item;
            this.jumlahItem++;
        }
    }

    public void tampilkanPesanan() {
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + this.nomorMeja);
        for (int i = 0; i < this.jumlahItem; i++) {
            System.out.println((i + 1) + ". " +
this.daftarItem[i].getNamaItem() + " - Rp " +
this.daftarItem[i].getRincian().getHargaTotal() + " (Harga: " +
this.daftarItem[i].getRincian().getHargaDasar() + ", Pajak: " +
this.daftarItem[i].getRincian().getPajak() + ")");
        }
    }

    // Getter
```

```
    public int getNomorMeja() {
        return this.nomorMeja;
    }

    public ItemPesanan[] getDaftarItem() {
        return this.daftarItem;
    }

    public int getJumlahItem() {
        return this.jumlahItem;
    }

    // Setter
    public void setNomorMeja(int nomorMeja) {
        this.nomorMeja = nomorMeja;
    }

    public void setDaftarItem(ItemPesanan[] daftarItem) {
        this.daftarItem = daftarItem;
    }

    public void setJumlahItem(int jumlahItem) {
        this.jumlahItem = jumlahItem;
    }
}
```

Kasir

```
package unguided;

public class Kasir {
    private String namaKasir;

    // Constructor
    public Kasir(String namaKasir) {
        this.namaKasir = namaKasir;
    }

    // Behavior
    public void prosesPembayaran(Pesanan pesanan) {
        double totalSubtotal = 0;
        double totalPajak = 0;
        ItemPesanan[] items = pesanan.getDaftarItem();
        for (int i = 0; i < pesanan.getJumlahItem(); i++) {
            totalSubtotal += items[i].getRincian().getHargaDasar();
```

```
            totalPajak += items[i].getRincian().getPajak();
        }
        double totalBayar = totalSubtotal + totalPajak;

        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + this.namaKasir);
        System.out.println("Pesanan        : Meja " +
pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " +
pesanan.getJumlahItem());
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
        System.out.println("==============================");
    }

    // Getter
    public String getNamaKasir() {
        return this.namaKasir;
    }

    // Setter
    public void setNamaKasir(String namaKasir) {
        this.namaKasir = namaKasir;
    }
}
```

Main

```
package unguided;

public class Main {
    // Behavior
    public static void main(String[] args) {
        Kasir kasir = new Kasir("Budi");

        ItemPesanan item1 = new ItemPesanan("Iced Americano", 18000);
        ItemPesanan item2 = new ItemPesanan("Beef Croissant", 25000);
        ItemPesanan item3 = new ItemPesanan("Matcha Latte", 22000);

        Pesanan pesanan = new Pesanan(4, 5);
        pesanan.tambahItem(item1);
        pesanan.tambahItem(item2);
        pesanan.tambahItem(item3);
```

```
        pesanan.tampilkanPesanan();
        System.out.println();
        kasir.prosesPembayaran(pesanan);
    }
}
```

Output :

```
PS D:\Praktikum PBO\modul-05-relasi-antar-kelas>  d:; cd 'd:\Prak
Java\jdk-17\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessage
eStorage\479783380518d35d0662f1a127f9c8f1\redhat.java\jdt_ws\modu
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
============================
```

Penjelasan :

Poin utama dalam program ini adalah implementasi tiga tingkatan relasi antar objek:

- Komposisi (ItemPesanan memiliki RincianHarga)
  Merupakan hubungan kepemilikan mutlak (*part-whole* yang ketat). Objek RincianHarga diinstansiasi secara langsung menggunakan kata kunci new di dalam *constructor* kelas ItemPesanan. Hal ini menjamin bahwa siklus hidup rincian harga terikat penuh pada item pesanan; jika objek ItemPesanan musnah/dihapus, maka RincianHarga miliknya tidak dapat berdiri sendiri dan akan ikut musnah.
- Agregasi (Pesanan memiliki ItemPesanan)
  Merupakan hubungan *part-whole* yang lebih longgar. Kelas Pesanan menampung kumpulan ItemPesanan, namun objek ItemPesanan diinstansiasi secara mandiri di luar kelas Pesanan. Objek-objek item tersebut baru disatukan ke dalam wadah Pesanan melalui *method* tambahItem(). Jika objek Pesanan dibatalkan, objek ItemPesanan secara logis masih dapat eksis di memori.
- Asosiasi (Kasir menggunakan Pesanan)
  Merupakan hubungan penggunaan tanpa adanya kepemilikan bersyarat. Kelas Kasir tidak menyimpan objek Pesanan sebagai atribut di dalam kelasnya. Interaksi hanya terjadi sesaat ketika kasir memproses transaksi, di mana objek Pesanan

sekadar dilewatkan sebagai parameter pada *method* prosesPembayaran(Pesanan pesanan).

D. Kesimpulan

Berdasarkan praktikum Modul 5 (Relasi Antar Kelas), dapat disimpulkan bahwa dalam Pemrograman Berorientasi Objek (PBO), kelas-kelas tidak berdiri sendiri melainkan saling berinteraksi melalui tiga jenis relasi utama dengan tingkat keterikatan (coupling) yang berbeda:

1. Asosiasi : Merupakan bentuk relasi paling longgar di mana dua objek berinteraksi tanpa saling memiliki atau menyimpan referensi permanen. Objek hanya saling menggunakan melalui parameter pada method. Hal ini dibuktikan pada interaksi kelas Dosen yang memberi nilai ke Mahasiswa (guided), serta kelas Kasir yang memproses pembayaran objek Pesanan (unguided).
2. Agregasi : Merupakan relasi kepemilikan yang bersifat longgar di mana objek penampung (container) menyimpan kumpulan objek lain, namun objek yang ditampung tetap dapat berdiri sendiri secara independen di luar wadahnya. Hal ini diterapkan pada kelas Jurusan yang menampung daftar Mahasiswa (guided), serta kelas Pesanan yang menampung kumpulan ItemPesanan (unguided). Komposisi : Merupakan bentuk relasi kepemilikan yang sangat kuat di mana siklus hidup objek bagian terikat penuh pada objek induknya. Objek bagian dibuat langsung di dalam konstruktor objek utama dan tidak dapat hidup secara terpisah. Contohnya adalah kelas Komputer yang menginisiasi komponennya (Processor, Monitor, Mouse) (guided), serta kelas ItemPesanan yang secara otomatis menginisiasi objek RincianHarga di dalam konstruktornya (unguided).

Pemilihan jenis relasi yang tepat (Asosiasi, Agregasi, atau Komposisi) sangat penting dalam rekayasa perangkat lunak agar struktur program mampu mencerminkan kondisi operasional dunia nyata secara akurat, menjaga integritas data, serta mempermudah pemeliharaan sistem secara modular.